

Week 1: Cloud & AWS Fundamentals + Terraform Foundations

Cloud Engineering Internship Guide

Week 1: Cloud & AWS Fundamentals + Terraform Foundations

The Complete Theoretical Intern Training & Implementation Guide

Welcome to the Week 1 Cloud Engineering Intern Guide. Starting an internship in cloud engineering can feel overwhelming. You are introduced to abstract networks, security models, storage tiers, and Infrastructure as Code (IaC) paradigms all at once.

This guide is designed to be your self-contained reference manual. It contains no code snippets (except for step-by-step shell execution commands and architecture diagrams), focusing instead on explaining every single concept, system boundary, and design decision from first principles. By reading this guide, you will understand exactly what was built during Week 1, why it was designed that way, how the components interact, and how to replicate the architectures step-by-step.

Table of Contents

- Module 1: Cloud Computing Foundations
- Module 2: AWS Global Infrastructure & Shared Responsibility
- Module 3: Core AWS Services Overview
- Module 4: AWS Identity & Access Management (IAM)
- Module 5: Amazon Simple Storage Service (S3) Deep-Dive
- Module 6: Programmatic Storage & SDK Operations (Boto3)
- Module 7: Enterprise CDN Edge Architecture (CloudFront + WAF + Logging)
- Module 8: Infrastructure as Code (IaC) & Terraform Foundations
- Module 9: Amazon Elastic Compute Cloud (EC2) Mechanics
- Module 10: Hands-On Lab & Project Setup Guide
- Module 11: Deployment CLI Commands, Verification, & Troubleshooting Runbook
- Module 12: Week 1 Learning Conclusion

Module 1: Cloud Computing Foundations

1.1 What is Cloud Computing?

Historically, businesses had to purchase physical servers, lease space in data centers, set up cooling systems, connect power grids, and hire systems administrators just to run a single web application. This approach required massive upfront capital expenditure (CapEx) and created long delays before any application could go live.

Cloud computing abstracts this physical complexity. It is the on-demand delivery of compute power, database storage, applications, and networking services over the internet, managed by cloud providers (like Amazon Web Services). Instead of buying hardware, organizations lease capacity on a pay-as-you-go basis, shifting their costs to operational expenditure (OpEx).

1.2 Five Essential Characteristics of Cloud Computing

According to the National Institute of Standards and Technology (NIST), cloud computing is defined by five core characteristics:

- **On-Demand Self-Service:** Users can provision resources (like virtual servers or storage) automatically without requiring human interaction with the cloud provider.
- **Broad Network Access:** Services are available over the network and accessed through standard mechanisms (such as web browsers or API clients) across desktops, laptops, and mobile devices.
- **Resource Pooling:** The provider's computing resources are pooled to serve multiple customers (multi-tenancy) using a physical virtualization layer. Physical resources are dynamically assigned and reassigned according to demand, without the customer knowing the exact physical location of the hardware.

- **Rapid Elasticity:** Resources can be elastically provisioned and released—sometimes automatically—to scale rapidly outward or inward in response to traffic demand.
- **Measured Service:** Cloud systems automatically control and optimize resource use by leveraging a metering capability (such as tracking CPU hours, gigabytes stored, or active network bandwidth). Resource usage is reported transparently, providing clear visibility into consumption.

1.3 Cloud Service Models (IaaS, PaaS, SaaS)

Cloud computing services are divided into three service delivery models, representing the division of management responsibility between the customer and the provider:

SOFTWARE AS A SERVICE (SaaS)
[Managed by Provider]: Hardware, Hypervisor, OS, Runtime, Application
[Managed by Customer]: User Accounts and Settings
PLATFORM AS A SERVICE (PaaS)
[Managed by Provider]: Hardware, Hypervisor, OS, Runtime (Python/Node)
[Managed by Customer]: Application Code, Configuration Settings
INFRASTRUCTURE AS A SERVICE (IaaS)
[Managed by Provider]: Physical Servers, Network Cables, Hypervisor
[Managed by Customer]: OS, Installed Runtimes, Firewall, App Code

- **Infrastructure as a Service (IaaS):**
- Provides raw networking, physical or virtual computers, and storage. The cloud provider manages the physical data centers, networking hardware, and the virtualization hypervisor layer.

- Your Responsibility: You choose and maintain the operating system, configure network firewalls, install application runtimes, and manage all security patches.
- AWS Example: Amazon EC2, Amazon VPC.
- Platform as a Service (PaaS):
- Eliminates the need for you to manage the underlying operating system and hardware. The cloud provider handles OS updates, patching, load balancing, and runtime environments.
- Your Responsibility: You write the application code and configure specific application settings.
- AWS Example: AWS Elastic Beanstalk, AWS Lambda (Serverless PaaS/FaaS), Amazon RDS.
- Software as a Service (SaaS):
- A complete, end-user software application managed entirely by the service provider.
- Your Responsibility: You only manage user accounts and application configurations.
- Example: Google Workspace, Salesforce, Microsoft 365.

1.4 Cloud Deployment Models

- Public Cloud: Owned and operated by a third-party cloud provider (like AWS). Physical hardware is shared among multiple organizations (tenants), but logically isolated.
- Private Cloud: Cloud infrastructure dedicated to a single organization. It can be built in-house or hosted by a third-party provider, providing maximum security control.
- Hybrid Cloud: Connects private, on-premises infrastructure with a public cloud. This allows sensitive data to stay on-premises while using the public cloud for burst capacity.

- Multi-Cloud: Using multiple public cloud providers (e.g., combining AWS and Google Cloud) to avoid vendor lock-in and optimize service selection.

Module 2: AWS Global Infrastructure & Shared Responsibility

2.1 Regions, Availability Zones, and Edge Locations

The AWS global footprint is built hierarchically to ensure high availability, redundancy, and low latency:

- AWS Regions: A physical geographic location anywhere in the world where AWS clusters data centers. Regions are completely isolated from one another to prevent disaster correlation (e.g., if one region loses power, it does not affect others). Regions are connected by a private, high-speed fiber-optic network.
- Availability Zones (AZs): Inside each Region, AWS builds multiple Availability Zones (typically 3 or more). An AZ consists of one or more physical data centers. Each AZ has redundant, independent power grids, cooling systems, and network connections. They are separated by physical distance (usually several miles) to prevent localized disasters (floods, fires, earthquakes) from taking down multiple zones, yet they remain connected via ultra-low latency fiber.
- Edge Locations / Point of Presence (PoPs): These are small, specialized data centers distributed globally near major population centers. They do not run full virtual servers; instead, they cache static content (like HTML pages, images, and videos) close to end-users using Amazon CloudFront (Content Delivery Network). This dramatically reduces latency.
- Local Zones & Wavelength Zones:
- Local Zones: Place compute, storage, and database services close to large population centers, allowing you to run applications requiring single-digit millisecond latency.

- Wavelength Zones: Embed AWS services directly within telecommunication providers' 5G networks, allowing developers to build applications for mobile devices with ultra-low latency.

2.2 AWS Shared Responsibility Model

When deploying workloads to the cloud, security is shared between AWS and the customer:

- AWS's Responsibility (Security "OF" the Cloud):
- AWS is responsible for protecting the infrastructure that runs all of the services offered in the AWS Cloud. This includes the physical security of data centers, the power grids, the physical networking switches, the server racks, and the hypervisor software layer that runs virtual machines.
- Customer's Responsibility (Security "IN" the Cloud):
- The customer is responsible for how they use the provisioned services. This includes configuring the guest operating system, keeping patches updated, establishing firewall rules (Security Groups), configuring IAM users and passwords, encrypting data at rest and in transit, and securing application code.

Module 3: Core AWS Services Overview

During Week 1, you will encounter the main services of the AWS ecosystem:

- Amazon EC2 (Elastic Compute Cloud): Virtual servers (compute instances) that you can launch, configure, and manage. It represents the IaaS model.
- Amazon ECS (Elastic Container Service): A highly scalable container management service used to run, stop, and manage Docker containers on a cluster.
- Amazon S3 (Simple Storage Service): Object storage built to store and retrieve any amount of data from anywhere on the web.

- AWS Lambda: A serverless compute service that runs your code in response to events (e.g., file uploads, API requests) without provisioning or managing servers.
- Amazon RDS (Relational Database Service): A managed service that makes it easy to set up, operate, and scale relational databases (like MySQL or Postgres) in the cloud.
- Amazon DynamoDB: A fully managed NoSQL database service that provides fast, predictable performance with seamless scalability.
- Amazon API Gateway: A fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale.
- AWS IAM (Identity & Access Management): The security framework that controls authentication (who can log in) and authorization (what permissions they have).
- Amazon CloudWatch: A monitoring and management service that collects performance logs, metrics, and alarms across all your AWS resources.

Module 4: AWS Identity & Access Management (IAM)

4.1 Users, Groups, and Roles

IAM controls access to AWS resources. It defines identities to govern permissions:

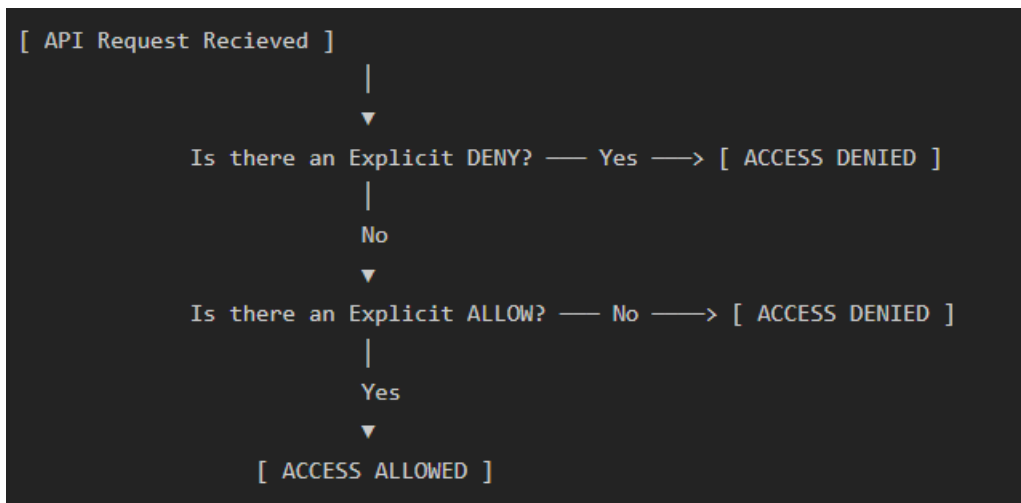
- The Root User: The single login identity created when the AWS account is first registered. It has absolute administrative access to all billing and technical resources.
- Best Practice: Lock the Root User immediately with a hardware MFA token, generate no API access keys for it, and only use it for billing modifications. All daily engineering operations should be performed by IAM Users with restricted rights.
- IAM Users: Logically isolated identities within the AWS account representing a single person or a non-human service worker. They can connect using:

- Console Access: A username and password.
- Programmatic Access: An Access Key ID and Secret Access Key used by SDKs, CLI, or IaC tools to authenticate API calls.
- IAM Groups: Logically grouped IAM users. By attaching permissions to a group, all users in that group inherit those permissions. This simplifies user management.
- IAM Roles: An identity that can be assumed temporarily by an authorized entity (an IAM User, an EC2 virtual machine, or an AWS service like Lambda). Roles do not have permanent passwords or access keys. Instead, when a resource assumes a role, AWS Security Token Service (STS) issues temporary, short-lived security credentials (valid for a few minutes to hours) that are rotated automatically.

4.2 Policies, Permissions, and Evaluation Logic

An IAM Policy is a document that explicitly grants or denies permissions. AWS evaluates these rules dynamically when an API call is made:

- Identity-Based Policies: Policies attached directly to an IAM User, Group, or Role. They specify what the identity can do.
- Resource-Based Policies: Policies attached directly to a resource (such as an S3 Bucket Policy). They specify who is allowed to access that resource.
- IAM Evaluation Policy Flow:
 - By default, all requests are Denied (Default Deny).
 - If an Explicit Deny is found in any applicable policy, the request is instantly denied, bypassing all other rules.
 - If an Explicit Allow is found and no Deny is present, the request is allowed.
 - If no policy explicitly allows the action, the request is denied (Implicit Deny).



Module 5: Amazon Simple Storage Service (S3) Deep-Dive

5.1 Object Storage vs. Block/File Storage

Traditional servers use Block Storage (like virtual hard drives) or File Storage (like shared folders). These systems organize data in local hierarchies of folders and sectors. To read a file, the OS must search the file tree, which gets slower as database sizes grow.

Amazon S3 uses Object Storage. Data is stored as independent entities (objects) within flat directories called Buckets. There are no nested folders at the physical disk level. S3 acts as a massive key-value database:

- Key: The unique text string identifying the object (e.g., `images/profile.jpg`).
- Value: The raw binary data of the file.
- Metadata: Key-value pairs containing information about the file (e.g., file size, upload date, custom tags, or Content-Type headers).

5.2 S3 Bucket Configuration & Security Mechanics

To keep data secure and organized, S3 uses several configuration layers:

- S3 Block Public Access: A safety mechanism that overrides bucket policies. When enabled, it blocks any public access policies from being attached to the bucket, protecting against accidental exposure.

- S3 Bucket Policies: Resource-based JSON policies that restrict access to objects within a bucket based on IP address, IAM User, or referring domain name.
- Access Control Lists (ACLs): Legacy access controls that manage permissions at the individual object level.
- Best Practice: Disable ACLs (Bucket Owner Enforced) and use IAM policies and Bucket Policies exclusively for modern access control.

5.3 S3 Versioning & Object Lock

- S3 Versioning: Versioning allows you to keep multiple versions of an object in the same bucket. When enabled, uploading an object with an existing key creates a new version instead of overwriting the previous file. Deleting an object places a "Delete Marker" on the object; the previous version can still be recovered. This provides robust protection against accidental deletions and modifications.
- S3 Object Lock (WORM Security Model): Object Lock enforces Write Once, Read Many (WORM) protection. It blocks objects from being deleted or overwritten for a fixed retention period. There are two main retention modes:
- Governance Mode: Users cannot overwrite or delete an object version unless they have special IAM permissions (`s3:BypassGovernanceRetention`).
- Compliance Mode: The retention period cannot be bypassed by anyone, including the root user.

5.4 S3 Lifecycle Policies

Lifecycle policies automate the transition and cleanup of objects to save costs:

- Transition Rules: Move objects to cheaper storage classes (e.g., Standard-IA or Glacier) based on age.
- Expiration Rules: Automatically delete objects after a specified number of days.
- S3 Storage Classes:
- Standard: High durability, availability, and performance object storage for frequently accessed data.

- Standard-IA (Infrequently Accessed): Lower storage cost but retrieval fees apply. Suitable for data accessed less than once a month.
- Glacier Flexible Retrieval / Deep Archive: Archive storage classes with retrieval times ranging from minutes to hours. Extremely cheap storage pricing.

Module 6: Programmatic Storage & SDK Operations (Boto3)

Developers use programmatic interfaces to manage AWS resources within applications. For Python developers, this is handled by Boto3.

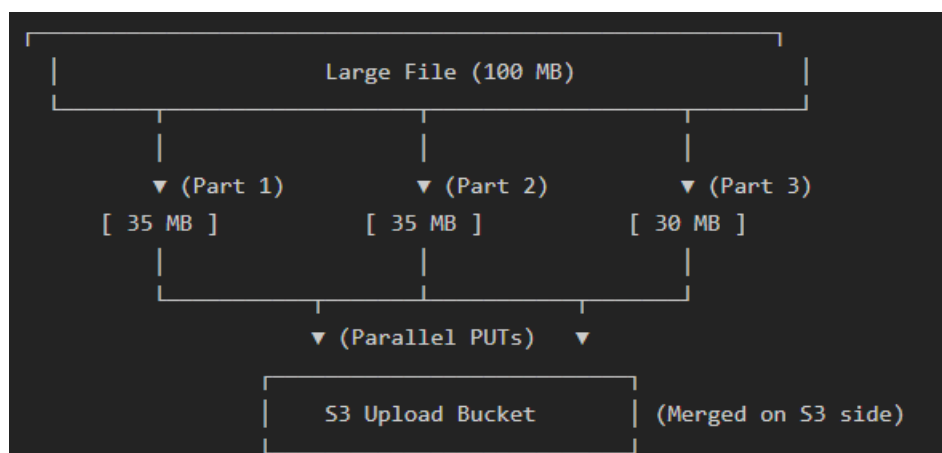
6.1 SDK Clients vs. Resources

Boto3 provides two interfaces:

- Client (Low-level): Provides a 1-to-1 mapping to the underlying AWS REST API. It returns raw JSON dictionary responses and requires manual parsing, but supports every AWS feature.
- Resource (High-level): An object-oriented abstraction that hides raw API requests behind Python objects (e.g., `s3.Bucket('my-bucket').objects.all()`). It is easier to read, but does not support all features of every service.

6.2 Multipart Upload Mechanics

When uploading large files (>100 MB) to S3, a single network interruption can fail the entire upload, forcing you to start over. S3 solves this with Multipart Uploads:



- Initiation: The SDK requests a Multipart Upload ID from S3.
- Chunking: The file is split into independent parts (each part must be 5 MB, except the final part).
- Parallel Upload: The parts are uploaded to S3 concurrently in separate threads. If one part fails, only that part needs to be re-uploaded.
- Completion: Once all parts are uploaded, the client sends a complete command, and S3 merges the parts into a single object.

6.3 Pre-Signed URLs and Signature Version 4 (SigV4)

Usually, S3 buckets are kept private to protect data. However, a web application might need to allow users to upload or download files directly from S3 without sharing IAM credentials.

A Pre-Signed URL grants temporary access using cryptographic signatures:

- The backend application uses its AWS IAM permissions to generate a URL.
- The URL includes parameters (Access Key, Timestamp, Expiry, and a Signature) calculated using AWS Signature Version 4 (SigV4).
- The signature validates that the request was generated by an authorized user.
- The client browser uses this URL to perform a GET or PUT operation. S3 validates the signature and processes the file if the URL has not expired.

6.4 Data Integrity via Content-MD5 Checked Uploads

During network transfers, files can experience silent data corruption (flipped bits).

To prevent this, pre-signed upload URLs can enforce Content-MD5 validation:

- Before generating the upload link, the client calculates the file's MD5 Hash and encodes it in Base64.
- This hash is embedded into the pre-signed URL signing parameters.
- When uploading the file, the client must include the `Content-MD5` header.

- S3 calculates the MD5 hash of the received payload. If it doesn't match the signature's hash, S3 rejects the upload with a `SignatureDoesNotMatch` or `InvalidDigest` error.

Module 7: Enterprise CDN Edge Architecture

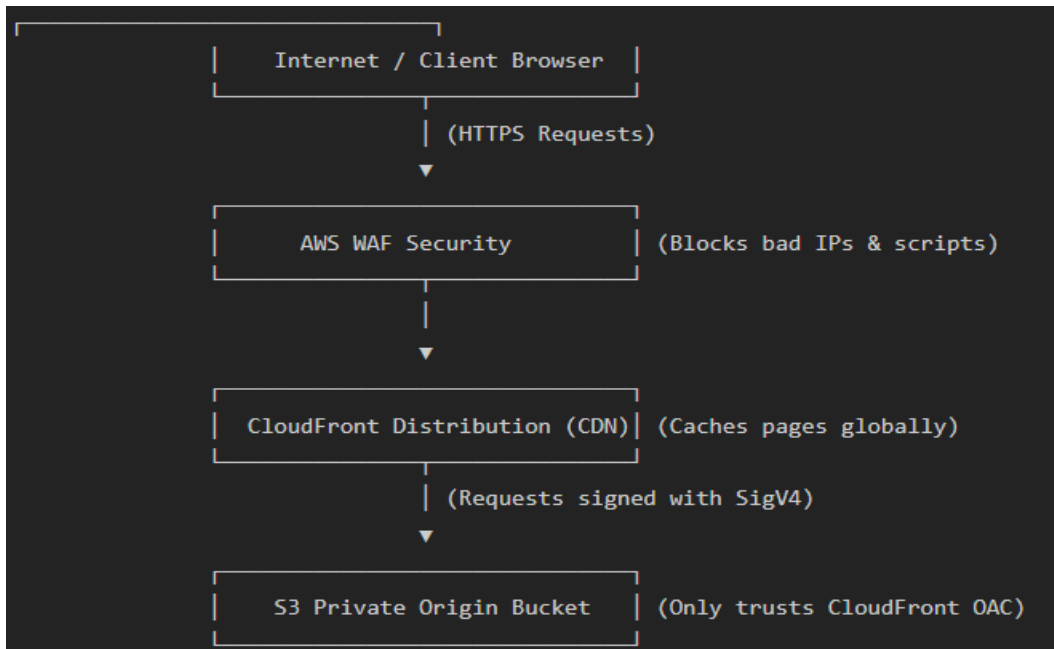
Serving assets directly from an S3 bucket is simple, but is not suitable for production deployments at scale.

7.1 Why S3 Static Hosting is Inadequate for Production

- **SSL/TLS Limitations:** S3 website endpoints only support HTTP. You cannot use a custom SSL certificate (HTTPS) directly on S3 endpoints (e.g., `https://www.yourdomain.com`).
- **High Latency:** If your bucket is in `us-east-1`, users in India or Europe must route traffic across the ocean, causing slow load times.
- **Higher Egress Costs:** Direct S3 data transfer is more expensive than routing through a Content Delivery Network (CDN).
- **DDoS Vulnerability:** S3 buckets are subject to request-rate limits and do not have built-in edge firewalls to block malicious traffic.

7.2 The Secure Solution: Origin Access Control (OAC)

To secure the site, we block all public access to the S3 bucket and route traffic through Amazon CloudFront:



- Global Caching: CloudFront caches S3 assets across hundreds of edge locations worldwide, ensuring low-latency access for users.
- Origin Access Control (OAC): CloudFront cryptographically signs requests sent to S3. The S3 bucket policy is configured to trust only CloudFront's service principal for the specific distribution ARN. Anyone attempting to access S3 directly is blocked with a `403 Forbidden` error.
- AWS WAF Integration: A Web Application Firewall (WAF) sits in front of CloudFront to inspect HTTP requests. It protects against automated bots, SQL injection, Cross-Site Scripting (XSS), and limits how many requests an IP can send.
- Access Auditing: Access logging is configured at both levels: S3 records raw reads, and CloudFront logs edge requests to a central, private log bucket.

Module 8: Infrastructure as Code (IaC) & Terraform Foundations

Managing infrastructure manually via the AWS Web Console is prone to errors, hard to audit, and cannot be replicated reliably. Infrastructure as Code (IaC) solves this by defining infrastructure using code files.

8.1 Declarative vs. Imperative IaC

- Imperative (e.g., AWS CLI scripts): You write scripts detailing the exact steps to build the infrastructure (e.g., "create vpc, then create subnet, then build ec2"). If the script runs twice, it will fail unless you write complex check logic.
- Declarative (e.g., Terraform): You define the desired end-state of the infrastructure (e.g., "I want a VPC with 2 subnets and 1 EC2 instance"). Terraform inspects the current state of your cloud resources, calculates the difference, and applies only the required changes.

8.2 Providers, Resources, and Data Sources

- Providers: Plugins that translate Terraform configurations into API calls. Each cloud provider (AWS, Azure, GCP) has its own provider plugin.
- Resources: Infrastructure components you want to build and manage (e.g., ``aws_instance`` for virtual machines or ``aws_s3_bucket`` for storage).
- Data Sources: Read-only queries used to retrieve information from AWS that was created outside of Terraform (e.g., querying the active list of AWS Availability Zones or AMI IDs).

8.3 The Role of the Terraform State File (``terraform.tfstate``)

Terraform stores a record of your managed infrastructure in a JSON file called the State File.

- Mapping: The state file maps the logical resource names in your code (e.g., ``aws_instance.my_server``) to the actual physical IDs in AWS (e.g., ``i-09f18a221b03``).
- Performance: Instead of querying AWS APIs every time you run a command, Terraform reads the state file to quickly verify the current setup.
- State Locking: When working in teams, multiple developers running Terraform simultaneously could corrupt the state file. To prevent this, a DynamoDB database table is used to lock the state file during execution.

- Remote Backend: By default, the state file is stored locally. In team environments, it is stored in a secure, private S3 bucket with versioning enabled to prevent data loss.

Module 9: Amazon Elastic Compute Cloud (EC2) Mechanics

9.1 Virtualization & Hypervisor Abstraction

Amazon EC2 provides virtual servers running on shared physical hardware. AWS uses a hypervisor (such as the Nitro System) to slice a physical host server into logically isolated virtual machines (VMs), ensuring each instance has dedicated access to CPU, RAM, and networking.

9.2 AMIs, Instance Types, and Network Placement

- AMIs (Amazon Machine Images): Pre-configured templates containing the operating system, server configurations, and system tools. You must specify an AMI ID to launch an EC2 instance.
- Instance Families: EC2 instances are optimized for different workloads:
- T-Family (t2.micro / t3.small): Burstable instances suitable for development and testing. They earn CPU credits during quiet periods and spend them during traffic spikes.
- C-Family: Optimized for high-performance compute workloads.
- M-Family: General-purpose balanced instances.
- Network Placement: EC2 instances must reside within a specific Subnet inside an Amazon VPC. For the instance to be accessible over the internet, it must be launched in a Public Subnet with a Public IP address assigned, and routing configured through an Internet Gateway.

Module 10: Hands-On Lab & Project Setup Guide

This module details the complete directories, execution files, automated CI/CD pipelines, and configuration steps required to initialize and run the Week 1 infrastructures.

10.1 Project File Directory Structure

Below is the file tree representing the directories inside your workspace containing the code resources for this week's assignments:

```
c:\Users\Vaibhav.S.P\Downloads\TringApps Intern
├── .github/
│   └── workflows/
│       └── deploy.yml          # CI/CD Deployment Pipeline config
├── EC2/
│   ├── main.tf                # Root compute host setup
│   └── modules/
│       └── ec2_instance/
│           ├── main.tf        # Sub-module computing declaration
│           ├── variables.tf    # Sub-module input parameters
│           └── output.tf       # Sub-module returned parameters
├── IAM/
│   ├── main.tf                # Basic IAM user & attachment policies
│   ├── my_test_file.txt       # Test payload object to upload
│   └── portfolio-assignment/
│       ├── main.tf            # website bucket hosting config
│       └── index.html         # Resume website index page
├── S3/
│   ├── Cloudfront/
│   │   ├── main.tf            # Secured CDN site configuration
│   │   ├── variables.tf       # Variable definitions
│   │   ├── outputs.tf         # Outputs definition
│   │   └── index.html         # Website testing landing page
│   ├── advanced_s3/
│   │   ├── main.tf            # Lifecycle & object lock policies
│   │   ├── variables.tf       # Variables configuration
│   │   ├── outputs.tf         # Outputs definition
│   │   ├── multipart_logic.py  # Python script: Parallel chunks S3 upload
│   │   └── presigned_logic.py  # Python script: Signed PUT/GET url test
│   └── static-website-hosting/
│       ├── main.tf            # Secure static hosting with remote S3 backend
│       ├── variables.tf       # Configuration variables
│       ├── outputs.tf         # Outputs declaration
│       └── site/
│           ├── index.html     # Rich HTML/CSS portfolio code
│           └── profile.jpg     # Profile image file asset
```

10.2 Step-by-Step Implementation Runbook

Follow these steps to deploy and test the different components of Week 1:

Task A: Basic Tools & Credentials Initialization

Before starting with Terraform, ensure your terminal is connected to your AWS Account:

1. Open terminal (PowerShell or Bash) in the repository root.
2. Initialize environment credentials.

aws configure

3. Type in your **AWS Access Key ID**, **AWS Secret Access Key**, set Default Region Name to us-east-1, and Default Output Format to json.
4. Validate the connection is active:

aws sts get-caller-identity

Task B: Basic IAM & Local S3 Deployments

1. Navigate to the IAM testing directory:

cd IAM

2. Initialize the backend providers:

terraform init

3. Generate the plan file:

terraform plan -out=tfplan

4. Deploy the user, bucket, file, and read-only policies:

terraform apply tfplan

5. *Verification*: Log into the AWS Web Console under IAM, search for the user Tring-VaibhavSP-Terraform, select its Permission Policies tab, and verify that the custom inline policy named VaibhavS3ReadOnlyAccess is attached.

6. Destroy the resources:

terraform destroy -auto-approve

Task C: S3 Static Website Hosting (The Portfolio Site)

1. Navigate to the basic website assignment directory:

cd IAM/portfolio-assignment

2. Initialize and deploy:

terraform init

terraform plan -out=tfplan

terraform apply tfplan

3. *Verification*: Capture the output variable website_url printed in the terminal. Copy the HTTP URL (e.g. `http://vaibhav-static-portfolio-1612.s3-website-us-east-1.amazonaws.com`) and paste it into a web browser. The page should load and render the text: *"Hello, I'm Vaibhav S. P. Welcome to my first cloud-hosted portfolio!"*

4. Destroy the resources:

terraform destroy -auto-approve

Task D: Modularized EC2 Compute Instances

1. Navigate to the compute directory:

```
cd ../../EC2
```

2. Initialize and deploy:

```
terraform init
```

```
terraform plan -out=tfplan
```

```
terraform apply tfplan
```

3. *Verification:* Open the AWS EC2 Console, select the us-east-1 region, and check that a t2.micro instance is active, running, and has a Public IP address corresponding to the public-ip-address output variable printed in your shell.

4. Destroy the resources:

```
terraform destroy -auto-approve
```

Task E: Running Python Storage Scripts (Boto3)

To run the programmatic Boto3 tests:

1. Navigate to the advanced S3 directory:

```
cd ../S3/advanced_s3
```

2. Initialize the infrastructure containing the secure vault bucket:

```
terraform init
```

```
terraform plan -out=tfplan
```

```
terraform apply tfplan
```

3. Install required Python packages:

```
pip install boto3 requests
```

4. Run the pre-signed URL generator and tester script:

```
python presigned_logic.py
```

Expected Output: The script will output the pre-signed PUT URL, upload the text data, return status code 200 (Success), print the pre-signed GET URL, and output the downloaded content.

5. Run the multipart file uploader script:

```
python multipart_logic.py
```

Expected Output: The script will generate a 10MB test file containing random bytes, partition it into chunks, and return: Multipart upload completed successfully!.

6. Destroy the resources:

```
terraform destroy -auto-approve
```

Task F: Enterprise Static Hosting with WAF, CloudFront & CI/CD

This is the main project deployment. Follow these instructions carefully:

Step 1: Create the Remote State S3 Backend

We need to create the state S3 bucket using the AWS CLI first so Terraform has a remote repository to save state locks:

1. Run this command to create the bucket (replace <account-id> with your AWS ID):

```
aws s3api create-bucket --bucket vaibhav-static-website-tfstate-  
285977275740 --region us-east-1
```

2. Turn on bucket versioning:

```
aws s3api put-bucket-versioning --bucket vaibhav-static-website-tfstate-285977275740 --versioning-configuration Status=Enabled
```

3. Turn on bucket encryption:

```
aws s3api put-bucket-encryption --bucket vaibhav-static-website-tfstate-285977275740 --server-side-encryption-configuration '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'
```

4. Block public access:

```
aws s3api put-public-access-block --bucket vaibhav-static-website-tfstate-285977275740 --public-access-block-configuration BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Step 2: Configure variables and backend in main.tf

1. Open `S3/static-website-hosting/main.tf` and check that the backend "s3" configuration references your state bucket:
 - `bucket = "vaibhav-static-website-tfstate-285977275740"`
2. Open `S3/static-website-hosting/variables.tf` and check that the `bucket_name` matches your preferred naming prefix (e.g. `vaibhav-static-website`).

Step 3: Run the initial local apply

1. In your terminal, navigate to the project folder:

```
cd S3/static-website-hosting
```

2. Initialize the backend:

terraform init

3. Run the apply:

terraform plan -out=tfplan

terraform apply tfplan

4. *Verification:* Capture the CloudFront domain name printed in the output variables. Open it in a browser. The portfolio index page should load.

Step 4: Configure GitHub Actions CI/CD Pipeline

Now, we hook up the automated git deployment pipeline:

1. Log in to GitHub in a browser. Go to your repository settings (VaibhavSP-TringApps-Internship).
2. Navigate to **Settings** $\rightarrow$ **Secrets and variables** $\rightarrow$ **Actions**.
3. Create two new repository secrets:
 - **AWS_ACCESS_KEY_ID:** Paste your AWS programmatic user access key.
 - **AWS_SECRET_ACCESS_KEY:** Paste your secret key.
4. Open a local file editor, change a small section of text inside S3/static-website-hosting/site/index.html (e.g. modify the welcome header text).
5. In your command terminal, navigate to the repository root directory and commit/push your changes:

git add .

git commit -m "update: modify landing text for deployment verification"

git push origin main

6. Open your GitHub Repository in your browser, select the Actions tab, and monitor the running workflow named `Enterprise Production Deployment Pipeline`.

How to inspect the execution pipeline:

1. Click on the latest workflow run (e.g., `#1` or `update: modify landing text...`).

2. In the left-hand menu under Jobs, click on the deploy job.

3. You will see a command execution list. Click on any step (such as `Sync files to S3` or `Invalidate CloudFront Cache`) to expand and view the real-time build and deployment logs.

7. Once the pipeline completes (turns green), load the CloudFront URL. Verify that your modified text is immediately rendered. The pipeline has successfully synced your files and invalidated CloudFront's edge memory!

8. When finished, clean up the resources to avoid billing costs:

cd S3/static-website-hosting

terraform m destroy -auto-approve

Module 11: Deployment CLI Commands, Verification, & Troubleshooting

11.1 Step-by-Step Hands-on Deployment Runbook

Step 1: Pre-requisites & AWS CLI Setup

Before running Terraform, ensure the AWS CLI is installed and configured on your machine.

Verify tool installations

git --version

terraform version

aws --version

Configure your AWS CLI credentials

aws configure

Enter your AWS Access Key ID, Secret Access Key, region (us-east-1), and format (json) when prompted.

Step 2: Provision the Remote State S3 Bucket

Create a private S3 bucket to store the remote Terraform state file.

Get your AWS Account ID

aws sts get-caller-identity --query Account --output text

Create the S3 state bucket (replace <account-id> with your actual ID)

aws s3api create-bucket --bucket vaibhav-static-website-tfstate-285977275740 --region us-east-1

Enable bucket versioning to allow state file recovery

aws s3api put-bucket-versioning --bucket vaibhav-static-website-tfstate-285977275740 --versioning-configuration Status=Enabled

Enable default server-side encryption (AES256)

aws s3api put-bucket-encryption --bucket vaibhav-static-website-tfstate-285977275740 --server-side-encryption-configuration

```
'{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}
```

```
# Block all public access to the state bucket
```

```
aws s3api put-public-access-block --bucket vaibhav-static-website-tfstate-285977275740 --public-access-block-configuration BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Step 3: Deploy the Terraform Infrastructure

Navigate to the project folder and run the deployment commands.

```
# Change directory to the static website project
```

```
cd S3/static-website-hosting
```

```
# Initialize the Terraform workspace and download the provider plugins
```

```
terraform init
```

```
# Validate configuration syntax
```

```
terraform validate
```

```
# Review the changes Terraform will make
```

```
terraform plan -out=tfplan
```

```
# Apply the plan to build the S3 buckets, CloudFront distribution, and WAF rules
```

```
terraform apply tfplan
```

Step 4: Verify the Deployment

- Locate the CloudFront domain name printed in the Terraform outputs.
- Load the URL in your web browser (using HTTPS).
- Verify that your portfolio page displays correctly.
- Try loading the S3 bucket URL directly in your browser. You should receive a `403 Forbidden` error, proving that Origin Access Control (OAC) is successfully blocking direct access.

Step 5: Clean Up Resources

When you are finished testing, destroy the resources to avoid ongoing AWS charges.

Teardown the infrastructure

terraform destroy -auto-approve

11.2 Troubleshooting Common Issues

- Error: `BucketAlreadyExists`:
 - Cause: S3 bucket names must be globally unique across all AWS accounts. Another user has already claimed the bucket name in your config.
 - Resolution: Edit your `variables.tf` file and change the `bucket\_name` default value to a unique name containing your initials or a random number.
- Error: `DependencyViolation` during VPC Subnet Teardown:
 - Cause: When Lambdas or EC2 instances connect to resources inside private subnets, AWS creates Network Interfaces (ENIs) in those subnets. AWS caches these ENIs for 15–20 minutes after the resources are deleted, blocking subnet deletion.
 - Resolution: Do not cancel the destroy command. Terraform will continue retrying in the background. Once the AWS cache expires, the subnets will be deleted automatically.
- Error: Access Denied (`403 Forbidden`) via CloudFront:

- Cause: The S3 bucket policy allowing CloudFront access is missing, or the CloudFront distribution configuration has an incorrect origin configuration.
- Resolution: Run `terraform apply` again to ensure the S3 bucket policy is correctly applied, and verify that your `index.html` file is located in the root of your S3 bucket.

Module 12: Week 1 Learning Conclusion

- Throughout this foundational week, we have progressed from basic cloud concepts to building automated, secure pipelines. Here is a summary of the core competencies and learning outcomes achieved:
 - **Cloud Infrastructure Concepts:** Understood the architectural differences between IaaS, PaaS, and SaaS, the layout of AWS Regions and Availability Zones, and the importance of the Shared Responsibility Model in cloud environments.
 - **IAM Security Foundations:** Learned to apply the Principle of Least Privilege by configuring custom IAM Policies, managing users, and defining roles to restrict resource access.
 - **Amazon S3 Features:** Explored S3 configuration options, including Versioning for backup, Object Lock (Governance/Compliance) for data security, and Lifecycle configurations to transition data to Glacier to optimize storage costs.
 - **Programmatic AWS Interaction:** Programmed Python scripts using Boto3 to manage data transfers using Multipart parallel chunk uploads and Pre-Signed URLs with MD5 verification to protect data in transit.
 - **Enterprise Static Web Hosting:** Designed and deployed a secure website delivery pipeline using S3 for private storage, CloudFront for global caching, Origin Access Control (OAC) to restrict access, and AWS WAF for rate-limiting.

- **Infrastructure as Code (IaC) with Terraform:** Initialized local provider environments, created reusable modules, and configured remote backend storage in S3 with state locking to coordinate team changes.
- **CI/CD Automation:** Created a GitHub Actions workflow to automate the deployment process, sync assets, and invalidate CloudFront edge caches on git commits.
- By combining these skills, you have built a complete, secure infrastructure pipeline, establishing a strong foundation for the rest of your internship!

- **Appendix A: Glossary of Week 1 Terminology**

Acronym / Term	Full Name	Definition / Cloud Role
IaaS	Infrastructure as a Service	Cloud model providing raw virtual compute, storage, and networking (e.g., Amazon EC2).
PaaS	Platform as a Service	Cloud model providing managed execution runtimes without operating system control (e.g., AWS Elastic Beanstalk).
SaaS	Software as a Service	Fully managed end-user software hosted in the cloud (e.g., Salesforce, Microsoft 365).
IAM	Identity & Access Management	The framework used to manage access identities, groups, policies, and permissions in AWS.
ARN	Amazon Resource Name	A standardized string format used to uniquely identify any AWS resource globally across all accounts and regions.
S3	Simple Storage	Managed object storage service designed

Acronym / Term	Full Name	Definition / Cloud Role
	Service	for highly scalable, flat storage of files.
OAC	Origin Access Control	A security mechanism that restricts S3 bucket access strictly to requests routed through Amazon CloudFront.
WAF	Web Application Firewall	Security layer protecting HTTP resources against Layer-7 exploits (DDoS, SQL injections, rate limits).
IaC	Infrastructure as Code	The practice of provisioning and managing cloud infrastructure via configuration files (e.g., Terraform).

- **Appendix B: Week 1 Troubleshooting & Debugging**

Error Message / Issue	Likely Cause	Step-by-Step Resolution
AccessDenied during terraform apply	Local terminal credentials do not have the required IAM permissions to provision resources.	Check aws sts get-caller-identity to see which user is active. Verify the IAM user has the AdministratorAccess policy attached in the console.
BucketAlreadyExists during deployment	The S3 bucket name is already occupied by another user in the global AWS namespace.	Edit variables.tf, change the default value for the bucket name variable to a unique identifier (e.g., append your initials or a timestamp), then run terraform apply again.
403 Forbidden on CloudFront URL	1. S3 Bucket Policy is missing OAC read	1. Verify the S3 bucket policy contains

Error Message / Issue	Likely Cause	Step-by-Step Resolution
	statements. 2. Object is not uploaded. 3. CloudFront configuration points to the wrong S3 origin.	the AllowCloudFrontServicePrincipal statement. 2. Verify index.html is present in the S3 bucket. 3. Confirm origin config in main.tf points to the correct bucket resource.
ExpiredToken on AWS CLI commands	Temporary AWS single sign-on (SSO) credentials or session tokens have expired.	Re-authenticate your local workstation by running aws sso login (if using SSO) or replace the AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and AWS_SESSION_TOKEN in your environment.
Error acquiring state lock	Another process is currently running Terraform, or a previous run crashed without releasing the lock.	Verify no other terminal is running Terraform. If sure it crashed, run terraform force-unlock <LOCK_ID> using the Lock ID printed in the error message.